

Reviewer Pack — Fiedler Participation Entropy Lower-Bounds Tseitin Tree-Reso...

Entry 4a5a7ca02658 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-18 13:00:28 UTC

Fiedler Participation Entropy Lower-Bounds Tseitin Tree-Resolution Size

Entry ID: 4a5a7ca02658 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-18 13:00:28 UTC

1. Conjecture

Field A (mathematical branch): Anderson localization theory on finite graphs — Rényi-2 participation entropy / inverse participation ratio (IPR) of Laplacian eigenvectors (Anderson 1958; Aizenman-Molchanov 1993; Erdős-Schlein-Yau 2009; Bauerschmidt-Knowles-Yau 2017 ‘Local Kesten-McKay law and complete delocalization for random regular graphs’; He-Knowles 2021). The functional $\text{IPR}(G) := \sum_v \varphi_2(v)^4$ of the unit Fiedler eigenvector φ_2 of L_G is an L^{4/L_2} spectral statistic measuring eigenmode spatial concentration; it uses real fourth-power arithmetic and admits NO polynomial F_q extension (since $|\cdot|^4$ vs $|\cdot|^2$ ratios are not preserved under polynomial ring lifts), Aaronson-Wigderson algebrization-safe. Distinct from blacklisted Tseitin attempts: signed-Laplacian holonomy is the gauge-invariant smallest eigenvalue of L^σ on a $\mathbb{Z}/2$ line bundle; effective resistance is a pseudoinverse diagonal pairing; Szegedy phase gap is a singular value on the edge-orientation space; $p=3$ modulus is a Hölder flow-cost program; none is the L^4 mass of a Laplacian eigenvector. arXiv ‘participation ratio’ AND (‘Tseitin’ OR ‘resolution refutation’ OR ‘proof complexity’) returns 0 direct hits with <5 adjacent papers on Anderson delocalization on random regular graphs. **Field B** (complexity object): Tree-like Resolution / DPLL refutation size $t(T(G,\omega))$ for Tseitin XOR formulas $T(G,\omega)$ on connected 3-regular graphs G with odd charge $\omega : V(G) \rightarrow \mathbb{F}_2$ (Urquhart 1987; Ben-Sasson-Wigderson 2001; Alekhnovich-Razborov 2001 expansion regime), in the Cook-Reckhow-Krajíček bounded-arithmetic framework where $\neg T(G,\omega)$ is Σ^b_0 and $\log_2 t(T(G,\omega))$ lower-bounds V^0 -witnessing complexity.

Statement:

For every connected 3-regular graph G on n vertices and odd charge $\omega : V(G) \rightarrow \mathbb{F}_2$, let $\lambda_2(G)$ be the second-smallest eigenvalue of the combinatorial Laplacian L_G and let φ_2 be a unit eigenvector for λ_2 ; define the Fiedler IPR by $\text{IPR}(G) := \sum_v \varphi_2(v)^4$ (in the degenerate case, minimum over unit eigenvectors of the λ_2 -eigenspace). We conjecture that there is an absolute constant $c > 0$ such that for every such (G,ω) , $\log_2 t(T(G,\omega)) \geq c \cdot \lambda_2(G) / \text{IPR}(G)$, where $t(T(G,\omega))$ is the minimum tree-like Resolution refutation size. A single (G,ω) with $\log_2 t^*(T(G,\omega)) < 0.05 \cdot \lambda_2(G) / \text{IPR}(G)$ refutes the conjecture.

Rationale (proposer’s reasoning):

Ben-Sasson-Wigderson Tseitin lower bounds rest on edge-expansion $h(G)$; Cheeger ties h to λ_2 , but the standard $\sqrt{2\lambda_2} \geq h$ bound is loose unless the Fiedler vector is GLOBALLY DELOCALIZED — Anderson-style — across many vertices. Localization (small $1/\text{IPR} \approx O(1)$) signals a low-frequency mode concentrated on a small cut that DPLL can exploit

by branching on the bottleneck edges; multiplying λ_2 by the effective support $1/\text{IPR}$ sharpens the spectral component to count how ‘global’ the Fiedler mode actually is. The invariant is structural on (G, ω) (two Tseitin formulas both $\equiv \text{FALSE}$ can have very different $\text{IPR} \cdot \lambda_2$), shielding from Razborov-Rudich, and uses non-ring real arithmetic, shielding from algebrization.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cf9ab63de2a8f6b9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all 900 trials (5 sizes $\times$ 30 random 3-regular seeds + 5 planted-bottleneck graphs/seed = 750+150), compute $R(G, \omega) = \log_2(\text{DPLL decisions}) / (\lambda_2 / \text{IPR})$. SUPPORTED iff $\min R \geq 0.05$ over every trial; FALSIFIED iff any single (G, ω) yields $R < 0.05$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.86	SAFE	SAFE
KARP_LIPTON	SAFE	0.92	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Tseitin tree resolution lower bound Fiedler eigenvector participation ratio - Laplacian spectral gap inverse participation ratio proof complexity Tseitin - tree-like resolution refutation size expander eigenvector localization IPR

Top relevant hits considered: - [<http://arxiv.org/abs/0906.0693v3>] An improved lower bound on the counterfeit coins problem - [<http://arxiv.org/abs/1007.1875v2>] Lower Bounds for Quantum Oblivious Transfer - [<http://arxiv.org/abs/1801.08709v2>] Adaptive Lower Bound for Testing Monotonicity on the Line - [<http://arxiv.org/abs/math/0211247v1>] Inverse spectral problems for Sturm-Liouville operators with singular potentials - [<http://arxiv.org/abs/1010.0094v1>] An Inverse Spectral Theorem - [<http://arxiv.org/abs/math/0611745v1>] The inverse spectral problem for the discrete cubic string

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
```

```

from collections import defaultdict

def generate_3_regular_graph(n):
    if n % 2 != 0:
        raise ValueError("n must be even for 3-regular graph")
    vertices = list(range(n))
    edges = []
    remaining_degrees = {v: 3 for v in vertices}
    while any(d > 0 for d in remaining_degrees.values()):
        u = random.choice([v for v in vertices if remaining_degrees[v] > 0])
        v = random.choice([v for v in vertices if remaining_degrees[v] > 0 and v != u and (u, v) not in edges])
        edges.append((u, v))
        remaining_degrees[u] -= 1
        remaining_degrees[v] -= 1
    return edges

def generate_planted_bottleneck_graph(n):
    if n < 10:
        raise ValueError("n must be at least 10 for planted bottleneck")
    k = n // 2
    left_vertices = list(range(k))
    right_vertices = list(range(k, n))
    left_edges = generate_3_regular_graph(k)
    right_edges = generate_3_regular_graph(k)
    bridge_edges = [(random.choice(left_vertices), random.choice(right_vertices)) for _ in range(2)]
    edges = left_edges + right_edges + bridge_edges
    return edges

def matrix_multiply(A, B):
    n = len(A)
    m = len(B[0])
    p = len(B)
    result = [[0 for _ in range(m)] for _ in range(n)]
    for i in range(n):
        for j in range(m):
            for k in range(p):
                result[i][j] += A[i][k] * B[k][j]
    return result

def matrix_subtract(A, B):
    n = len(A)
    m = len(A[0])
    result = [[0 for _ in range(m)] for _ in range(n)]
    for i in range(n):
        for j in range(m):
            result[i][j] = A[i][j] - B[i][j]
    return result

def matrix_transpose(A):
    n = len(A)
    m = len(A[0])
    result = [[0 for _ in range(n)] for _ in range(m)]
    for i in range(n):
        for j in range(m):
            result[j][i] = A[i][j]
    return result

```

```

def matrix_trace(A):
    n = len(A)
    trace = 0
    for i in range(n):
        trace += A[i][i]
    return trace

def matrix_determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    det = 0
    for col in range(n):
        minor = [row[:col] + row[col+1:] for row in A[1:]]
        det += ((-1) ** col) * A[0][col] * matrix_determinant(minor)
    return det

def matrix_inverse(A):
    n = len(A)
    det = matrix_determinant(A)
    if det == 0:
        raise ValueError("Matrix is not invertible")
    adjugate = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(n):
            minor = [row[:j] + row[j+1:] for row in (A[:i] + A[i+1:])]
            adjugate[j][i] = ((-1) ** (i + j)) * matrix_determinant(minor)
    inverse = [[adjugate[i][j] / det for j in range(n)] for i in range(n)]
    return inverse

def matrix_eigen(A):
    n = len(A)
    eigenvalues = []
    eigenvectors = []
    for _ in range(n):
        x = [random.random() for _ in range(n)]
        for _ in range(100):
            Ax = matrix_multiply(A, [x])
            norm = math.sqrt(sum(xi ** 2 for xi in x))
            x = [xi / norm for xi in x]
        eigenvalue = sum(Ax[i] * x[i] for i in range(n))
        eigenvalues.append(eigenvalue)
        eigenvectors.append(x)
    return eigenvalues, eigenvectors

def compute_laplacian(edges, n):
    L = [[0 for _ in range(n)] for _ in range(n)]
    for u, v in edges:
        L[u][v] = -1
        L[v][u] = -1
        L[u][u] += 1
        L[v][v] += 1
    return L

def compute_fiedler_info(L):

```

```

eigenvalues, eigenvectors = matrix_eigen(L)
sorted_indices = sorted(range(len(eigenvalues)), key=lambda i: eigenvalues[i])
lambda_2 = eigenvalues[sorted_indices[1]]
phi_2 = eigenvectors[sorted_indices[1]]
IPR = sum(phi_2[i] ** 4 for i in range(len(phi_2)))
return lambda_2, IPR

def generate_tseitin_formula(edges, omega):
    clauses = []
    for u, v in edges:
        clauses.append([(u, 1), (v, 1), (u, v, 1)])
        clauses.append([(u, 0), (v, 0), (u, v, 0)])
    for v in omega:
        clauses.append([(v, omega[v])])
    return clauses

def tree_dp11(clauses, max_decisions=500000):
    assignments = {}
    decisions = 0
    def unit_propagate():
        nonlocal assignments
        changed = True
        while changed:
            changed = False
            for clause in clauses:
                unassigned = [lit for lit in clause if lit[0] not in assignments]
                if len(unassigned) == 1:
                    lit = unassigned[0]
                    if len(lit) == 2:
                        var, val = lit
                        if var not in assignments:
                            assignments[var] = val
                            changed = True
                else:
                    var1, var2, val = lit
                    if var1 in assignments and var2 in assignments:
                        if assignments[var1] == val and assignments[var2] == val:
                            assignments[(var1, var2)] = val
                            changed = True
    def backtrack():
        nonlocal assignments, decisions
        for var in list(assignments.keys()):
            if isinstance(var, tuple):
                del assignments[var]
        for var in list(assignments.keys()):
            if not isinstance(var, tuple):
                del assignments[var]
                break
    unit_propagate()
    while len(assignments) < len(set(lit[0] for clause in clauses for lit in clause if len(lit) == 2 and lit[0] not in assignments)):
        if decisions >= max_decisions:
            return max_decisions
        decisions += 1
        var = random.choice([lit[0] for clause in clauses for lit in clause if len(lit) == 2 and lit[0] not in assignments])
        assignments[var] = random.choice([0, 1])
        unit_propagate()

```

```

        satisfied = any(all(lit[0] in assignments and (len(lit) == 2 and assignments[lit[0]] == lit[1])
        if not satisfied:
            backtrack()
    return decisions

def run_trial(seed):
    random.seed(seed)
    n = random.choice([8, 10, 12, 14, 16])
    edges = generate_3_regular_graph(n)
    omega = {v: random.choice([0, 1]) for v in range(n)}
    L = compute_laplacian(edges, n)
    lambda_2, IPR = compute_fiedler_info(L)
    if IPR == 0:
        IPR = 1
    clauses = generate_tseitin_formula(edges, omega)
    decisions = tree_dp11(clauses)
    R = math.log2(decisions) / (lambda_2 / IPR) if lambda_2 != 0 and IPR != 0 else 0
    conjecture_holds = R >= 0.05
    counterexample = f"R={R}, decisions={decisions}, λ_2={lambda_2}, IPR={IPR}" if not conjecture_holds else ""
    return {
        "metric_name": "R(G,ω)",
        "metric_value": R,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
    trials = []
    for seed in seeds:
        trial = run_trial(seed)
        print(f"TRIAL: {trial}")
        trials.append(trial)
    metric_values = [trial["metric_value"] for trial in trials]
    mean = sum(metric_values) / len(metric_values)
    std = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = sum(trial["conjecture_holds"] for trial in trials) / len(trials)
    if all(trial["conjecture_holds"] for trial in trials):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif any(not trial["conjecture_holds"] for trial in trials):
        first_failing_seed = seeds[next(i for i, trial in enumerate(trials) if not trial["conjecture_holds"])]
        counterexample = next(trial["counterexample"] for trial in trials if not trial["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7da3a703.py", line 213, in <module>
    trial = run_trial(seed)
            ^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7da3a703.py", line 193, in run_trial
    lambda_2, IPR = compute_fiedler_info(L)
                    ^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7da3a703.py", line 126, in compute_fied
    eigenvalues, eigenvectors = matrix_eigen(L)
                                ^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7da3a703.py", line 111, in matrix_eigen
    eigenvalue = sum(Ax[i] * x[i] for i in range(n))
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7da3a703.py", line 111, in <genexpr>
    eigenvalue = sum(Ax[i] * x[i] for i in range(n))
                    ~~~~~^~~~~~
TypeError: can't multiply sequence by non-int of type 'float'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in `matrix_eigen` before any trials completed, so no R values were produced and the pre-registered support condition cannot be evaluated. | next: Fix the `matrix_eigen` bug (`Ax` appears to be a list of sequences rather than floats; ensure proper matrix-vector multiplication returns a flat numeric vector) and rerun all 900 trials.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	239087
2	preregistration	claude_max	opus	0	0	6271
3	novelty	claude_max	opus	0	0	3553
4	novelty	claude_max	opus	0	0	6019
5	test_gen	mistral	codestral-latest	0	0	318469
6	test_gen	mistral	codestral-latest	0	0	323251
7	test_gen	mistral	codestral-latest	0	0	325273
8	test_gen	mistral	codestral-latest	0	0	325451
9	judge	claude_max	opus	0	0	4396

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1551769 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 4}

(full prompt+response transcripts available in `research/audit/4a5a7ca02658.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4a5a7ca02658.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/4a5a7ca02658.tar.gz` (if generated)